

PL19
Backtracking: vincoli avanzati

Materiale ricostruito dalle slide ufficiali del corso.

Che problema stiamo risolvendo? Le slide studiano stringhe ternarie con conteggi ordinati, matrici con numero crescente di uni e permutazioni senza sottosequenza crescente di lunghezza 3.

Spiegazione intuitiva Qui la potatura non controlla soltanto se il prefisso è già invalido: calcola se esiste ancora abbastanza spazio per completarlo. Per le stringhe ternarie si contano i simboli minimi necessari per ottenere $\#a > \#b > \#c$. Per le matrici si usa il massimo numero di uni aggiungibile nelle righe rimanenti. Per le permutazioni si mantiene informazione sulle sequenze crescenti di lunghezza due, così si riconosce subito quando una nuova scelta creerebbe una LIS di lunghezza tre.

Cosa devi sapere

- Per stringhe con $a > b > c$ si calcola il minimo necessario per rendere completabile il prefisso.
- Per matrici si può usare il massimo numero di uni ottenibile dalle righe restanti.
- Per evitare LIS di lunghezza 3 si mantiene una struttura minimi e si controlla se una scelta chiude la sottosequenza.
- La complessità è espressa rispetto al numero $S(n)$ di soluzioni o N_v di nodi visitati.

Esercizi

1. Genera stringhe ternarie con $\#a > \#b > \#c$.
2. Genera matrici con numero di uni strettamente crescente per riga.
3. Genera permutazioni senza LIS di lunghezza 3.
4. Scrivi la potatura e dimostra che non elimina soluzioni valide.
5. Confronta enumerazione con verifica finale e backtracking potato.

Errore tipico da evitare Non confondere il numero di nodi visitati con il numero di soluzioni: la potatura può cambiare molto il primo.

Metodo di svolgimento Per ogni esercizio scrivi idea, pseudocodice, correttezza, complessità temporale, complessità spaziale, codice Python e test limite.

Fonte PL19.pdf: contenuto verificato sulle slide ufficiali collegate nella pagina del corso.